

# Sicurezza nelle reti

Giulia

7 maggio 2026

## 1 Race condition

Una Race Condition si verifica quando due o più processi/thread accedono contemporaneamente a una risorsa condivisa e il risultato finale dipende dall'ordine di esecuzione, che non è prevedibile né controllabile.

Se un programma privilegiato presenta una condizione di competizione, gli aggressori potrebbero essere in grado di influenzare l'output del programma privilegiato influenzando gli eventi incontrollabili.

Quando due thread di esecuzione simultanei accedono a una risorsa condivisa in un modo che produce involontariamente risultati diversi a seconda della tempistica dei thread o dei processi

```
function withdraw($amount)
{
    $balance = getBalance();
    if($amount <= $balance) {
        $balance = $balance - $amount;
        echo "You have withdrawn: $amount";
        saveBalance($balance);
    }
    else {
        echo "Insufficient funds.";
    }
}
```

Qui può verificarsi una Race Condition se ci sono due richieste di prelievo simultanee. Il controllo `if(amount <= balance)` viene superato da entrambi i thread prima che uno dei due aggiorni il saldo (es: due richieste di prelievo simultanee di €100 su un conto con saldo di €150, primo thread `getBalance()` → legge 150 secondo thread `getBalance()` → legge 150 entrambi calcolno 150 -100 e chiamano `saveBalance(50)` sono stati prelevati €200, ma il saldo mostra solo €50 di differenza)

### TOCTTOU – Time-Of-Check To Time-Of-Use

Il TOCTTOU è un tipo speciale di Race Condition che si verifica quando intercorre un intervallo di tempo tra il momento in cui viene effettuato un controllo su una risorsa e il momento in cui quella risorsa viene effettivamente utilizzata.

Durante questa finestra temporale, un attaccante può modificare lo stato della risorsa dopo che il controllo è già avvenuto con esito positivo, ma prima che venga utilizzata. Il programma quindi opera su qualcosa di diverso da ciò che aveva verificato, con conseguenze potenzialmente gravi.

Es: un programma privilegiato controlla che un file sia sicuro, ma prima di aprirlo un attaccante lo sostituisce con un file dannoso. Il programma utilizzerà il file malevolo, credendo di aver già verificato la sua sicurezza.

Consideriamo un programma Set-UID (bit di un file eseguibile se attivato, chiunque lo lanci lo esegue con i privilegi del proprietario del file) di proprietà di root, in cui l'UID effettivo è root e l'ID dell'utente reale è **seed**. Il programma scrive in un file nella directory **/tmp**, scrivibile da tutti gli utenti del sistema.

l'utente seed lancia il programma, ma poiché esso è Set-UID root, il sistema operativo lo esegue come se fosse root. Quando `open()` apre il file, guarda l'UID effettivo (`root = 0`) e concede l'accesso a qualsiasi file del sistema, indipendentemente da chi lo abbia avviato.

```
if (!access("/tmp/X", W_OK)) {
    /* the real user has the write permission */
    f = open("/tmp/X", O_WRITE);
    write_to_file(f);
}
else {
    /* the real user does not have the write permission */
    fprintf(stderr, "Permission denied\n");
}
```

Poiché root può scrivere su qualsiasi file, il programma deve prima verificare che sia l'utente reale ad avere i permessi necessari sul file di destinazione. A questo scopo viene utilizzata la funzione `access()`, che controlla se l'ID utente reale dispone dei permessi di scrittura su **/tmp/X**.

Tuttavia, tra il momento in cui `access()` effettua il controllo e il momento in cui `open()` apre il file in scrittura, esiste una finestra temporale. La funzione `open()` non controlla l'ID dell'utente reale, bensì quello effettivo, che è 0 (root), quindi il file verrà aperto indipendentemente da chi sia il proprietario effettivo del file in quel momento.

Un attaccante può sfruttare questa finestra temporale: dopo che `access()` ha verificato il file originale con esito positivo, ma prima che `open()` lo apra, è possibile sostituire **/tmp/X** con un link simbolico che punta a un file di sistema protetto, come ad esempio **/etc/passwd**. Il programma, eseguendosi con i privilegi di root, scriverà su quel file senza alcuna ulteriore verifica, compromettendo l'integrità del sistema.

## Contromisure

- Operazioni atomiche : eliminare la finestra tra controllo e utilizzo
- Controllo e utilizzo ripetuti: per rendere difficile vincere la “gara”.
- Collegamento simbolico sticky anche se la directory è scrivibile da tutti, un utente può creare o eliminare solo i propri file

- Protezione: per impedire la creazione di collegamenti simbolici.
- Principi del privilegio minimo: prevenire i danni dopo che la gara è stata vinta dall'attaccante. Es. un programma non dovrebbe mantenere i privilegi di root per tutta la sua esecuzione, ma elevarli solo nel momento in cui servono e rilasciarli subito dopo.

### Operazioni atomiche

Un primo approccio concreto utilizza i flag `O_CREAT | O_EXCL` combinati nella chiamata `open()`:

```
f = open(file, O_CREAT | O_EXCL);
```

Questi due flag insieme istruiscono `open()` ad aprire il file solo se non esiste già: in caso contrario, l'operazione fallisce. Poiché controllo e apertura avvengono nella stessa chiamata di sistema, il kernel garantisce l'atomicità dell'intera operazione, senza lasciare alcuna finestra temporale sfruttabile dall'attaccante.

Un secondo approccio, puramente teorico, immagina l'esistenza di un flag `O_REAL_USER_ID`:

```
f = open(file, O_WRITE | O_REAL_USER_ID);
```

Questo flag istruirebbe `open()` a verificare i permessi utilizzando l'UID reale anziché quello effettivo, affidando a una singola chiamata sia il controllo che l'utilizzo.

### Controllo e utilizzo ripetuti

Un secondo approccio consiste nel ripetere più volte la sequenza di controllo e utilizzo all'interno dello stesso programma. Come mostrato nel codice, il ciclo `access()` e `open()` viene eseguito tre volte, producendo tre file descriptor `fd1`, `fd2` e `fd3`. Al termine, il programma verifica tramite `fstat()` che i tre file descriptor puntino allo stesso inode:

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>

int main()
{
    struct stat stat1, stat2, stat3;
    int fd1, fd2, fd3;

    if (access("tmp/XYZ", O_RDWR)) {
        fprintf(stderr, "Permission denied\n"); 1
        return -1;
    }
}
```

```
if(stat1.st_ino == stat2.st_ino && stat2.st_ino == stat3.st_ino) {
    write_to_file(fd1);
}
```

```

else fd1 = open("/tmp/XYZ", O_RDWR);
if (access("/tmp/XYZ", O_RDWR)) {
    fprintf(stderr, "Permission denied\n");
    return -1;
}
else fd2 = open("/tmp/XYZ", O_RDWR);
if (access("/tmp/XYZ", O_RDWR)) {
    fprintf(stderr, "Permission denied\n");
    return -1;
}
else fd3 = open("/tmp/XYZ", O_RDWR);

// Check whether fd1, fd2, and fd3 has the same inode.
fstat(fd1, &stat1);
fstat(fd2, &stat2);
fstat(fd3, &stat3);

if(stat1.st_ino == stat2.st_ino && stat2.st_ino == stat3.st_ino) {
    // All 3 inodes are the same.
    write_to_file(fd1);
}
else {

```

← Window 2

← Window 3

← Window 4

← Window 5

2

3

Se gli inode non coincidono, significa che il file è stato modificato durante l'esecuzione e l'operazione viene bloccata. Per portare a termine un attacco riuscito, l'attaccante dovrebbe riuscire a sostituire `/tmp/XYZ` esattamente in ognuna delle cinque finestre temporali disponibili tra i vari controlli e aperture. La probabilità di riuscire a vincere la gara cinque volte consecutive è drasticamente inferiore rispetto a un programma con una sola condizione di competizione. Questa contromisura non elimina teoricamente la vulnerabilità, ma la rende nella pratica estremamente difficile da sfruttare.

### Protezione permanente dei collegamenti simbolici

I sistemi Linux moderni offrono una protezione permanente contro l'abuso dei collegamenti simbolici nelle directory scrivibili da tutti, come `/tmp`. Quando questa protezione è abilitata, un collegamento simbolico all'interno di una directory scrivibile da tutti può essere seguito solo se il proprietario del collegamento coincide con l'utente che lo sta seguendo, oppure con il proprietario della directory stessa. Questo impedisce all'attaccante di creare un link simbolico malevolo che punti a file di sistema protetti, poiché il programma privilegiato non sarà autorizzato a seguirlo.

```

// On Ubuntu 12.04, use the following:
$ sudo sysctl -w kernel.yama.protected_sticky_symlinks=1

// On Ubuntu 16.04, use the following:
$ sudo sysctl -w fs.protected_symlinks=1

```

Nel programma vulnerabile analizzato, l'UID effettivo del processo è root e la directory `/tmp` è anch'essa di proprietà di root. Con la protezione dei collegamenti simbolici abilitata, il sistema operativo permette di seguire un symlink solo quando il proprietario del collegamento corrisponde all'UID effettivo del processo che lo vuole seguire, oppure al proprietario della directory in cui si trova.

Nel nostro scenario, un attaccante con utente `seed` che crea un collegamento simbolico in `/tmp` ne diventa il proprietario. Quando il programma privilegiato

tenta di seguirlo, il sistema confronta il proprietario del link (**seed**) con l'EID del processo (**root**) e con il proprietario di **/tmp** (**root**): nessuna delle due condizioni è soddisfatta, quindi l'accesso viene negato. L'unico modo per creare un collegamento simbolico che il programma possa seguire sarebbe che fosse **root** stesso a crearlo, rendendo di fatto impossibile lo sfruttamento della vulnerabilità da parte di un utente non privilegiato.

| Follower (eUID) | Directory Owner | Symlink Owner | Decision (fopen()) |
|-----------------|-----------------|---------------|--------------------|
| seed            | seed            | seed          | Allowed            |
| seed            | seed            | root          | <b>Denied</b>      |
| seed            | root            | seed          | Allowed            |
| seed            | root            | root          | Allowed            |
| root            | seed            | seed          | Allowed            |
| root            | seed            | root          | Allowed            |
| root            | root            | seed          | <b>Denied</b>      |
| root            | root            | root          | Allowed            |

### Piccolo riassunto della terminologia

| Concetto                    | Valore (nel nostro caso) | Significato                                                          |
|-----------------------------|--------------------------|----------------------------------------------------------------------|
| UID reale                   | seed                     | Chi ha effettivamente lanciato il programma                          |
| UID effettivo (EID)         | root (0)                 | Con quali privilegi il processo sta girando                          |
| Proprietario del file       | root                     | Chi possiede il file eseguibile                                      |
| Bit Set-UID                 | attivo                   | Fa sì che il processo giri con i privilegi del proprietario del file |
| Proprietario di <b>/tmp</b> | root                     | Chi possiede la directory temporanea                                 |

| Funzione        | Controlla            | Conseguenza nel TOCTTOU                |
|-----------------|----------------------|----------------------------------------|
| <b>access()</b> | UID reale (seed)     | Controlla i permessi dell'utente reale |
| <b>open()</b>   | UID effettivo (root) | Apri qualsiasi file perché EID è root  |
| <b>fstat()</b>  | inode del file       | Verifica che il file non sia cambiato  |

## Contromisura: Principio del Privilegio Minimo

Il principio del privilegio minimo stabilisce che un programma non dovrebbe mai utilizzare più privilegi di quelli strettamente necessari all'operazione che sta svolgendo. Nel programma vulnerabile analizzato, il processo gira con EID **root** per tutta la sua esecuzione, anche durante operazioni che non richiedono privilegi elevati, come l'apertura di un file in **/tmp**.

La soluzione consiste nel ridurre temporaneamente i privilegi subito prima di aprire il file, impostando l'EID uguale al RID tramite **seteuid()**:

```
uid_t real_uid = getuid();
uid_t eff_uid = geteuid();
seteuid(real_uid);           // EID diventa seed
f = open("/tmp/X", O_WRITE);
seteuid(eff_uid);           // EID ripristinato a root
```

In questo modo, nel momento in cui `open()` viene eseguita, il processo ha i privilegi dell'utente reale e non di root. Anche se l'attaccante fosse riuscito a sostituire `/tmp/X` con un collegamento simbolico a un file di sistema protetto come `/etc/passwd`, l'apertura fallirebbe perché l'utente `seed` non dispone dei permessi necessari. I privilegi di root vengono ripristinati solo al termine dell'operazione, se necessari per le fasi successive del programma.

## Dirty COW Race Condition Attack

Caso interessante della vulnerabilità della race condition. Esistente nel kernel Linux da settembre 2007, è stato scoperto e attaccato nell'ottobre 2016. Interessa tutti i sistemi operativi basati su Linux, incluso Android.

### Conseguenze:

- Modifica i file protetti come `/etc/passwd`
- Ottieni i privilegi di root sfruttando la vulnerabilità.

## Mappatura della memoria con `mmap()`

`mmap()` è una chiamata di sistema che consente di mappare file o dispositivi direttamente in memoria. Il tipo di mappatura predefinito è la **mappatura supportata da file** (*file-backed mapping*): mappa un'area della memoria virtuale di un processo su un file, in modo che la lettura dall'area mappata corrisponda alla lettura del file.

### 1.1 Struttura del codice

Di seguito un esempio completo di utilizzo di `mmap()`:

Listing 1: Esempio di memory mapping con `mmap()`

```
int main()
{
    struct stat st;
    char content[20];
    char *new_content = "New-Content";
    void *map;

    // (1) Apre il file in modalita' lettura-scrittura
    int f = open("./zzz", ORDWR);
    fstat(f, &st);

    // (2) Mappa l'intero file in memoria
    map = mmap(NULL, st.st_size, PROT_READ | PROT_WRITE,
               MAP_SHARED, f, 0);

    // (3) Legge 10 byte dal file tramite la memoria mappata
```

```

memcpy((void*)content, map, 10);
printf("read: %s\n", content);

// (4) Scrive nel file tramite la memoria mappata
memcpy(map + 5, new_content, strlen(new_content));

// Pulizia
munmap(map, st.st_size);
close(f);
return 0;
}

```

## Argomenti di mmap()

La chiamata alla riga 2 ha la seguente firma:

```
void *mmap(void *addr, size_t length, int prot, int flags, int
          fd, off_t offset);
```

1. **Indirizzo iniziale** (addr): indirizzo di partenza per la memoria mappata. Se NULL, il kernel sceglie automaticamente l'indirizzo.
2. **Dimensione** (length): dimensione in byte della regione di memoria da mappare.
3. **Protezione** (prot): specifica se la memoria è leggibile e/o scrivibile (PROT\_READ, PROT\_WRITE). Deve essere compatibile con la modalità di apertura del file alla riga 1.
4. **Flag** (flags): determina se le modifiche alla mappatura sono visibili ad altri processi che mappano la stessa regione e se le modifiche vengono propagate al file sottostante (MAP\_SHARED o MAP\_PRIVATE).
5. **File descriptor** (fd): il file che deve essere mappato in memoria.
6. **Offset** (offset): indica da quale posizione all'interno del file deve iniziare la mappatura.

## Accesso tramite memcpy()

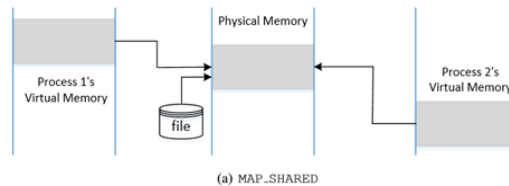
Una volta creata la mappatura, è possibile accedere al file con semplici operazioni di lettura e scrittura tramite `memcpy()`:

- Riga 3: legge 10 byte dal file tramite la memoria mappata.
- Riga 4: scrive nel file tramite la memoria mappata, a partire dall'offset 5.

## MAP\_SHARED

La memoria mappata si comporta come **memoria condivisa** tra i processi.

- Quando più processi mappano lo stesso file, possono utilizzare indirizzi di memoria *virtuale* diversi.
- L'indirizzo di memoria *fisica* in cui viene conservato il contenuto del file è però **lo stesso** per tutti i processi.
- Le modifiche effettuate da un processo sono immediatamente visibili agli altri e vengono propagate al file su disco.



## MAP\_PRIVATE

Il file viene mappato nella **memoria privata** del processo chiamante, con semantica *copy-on-write*:

- Le modifiche apportate alla memoria **non sono visibili** agli altri processi.
- Inizialmente, la memoria virtuale del processo punta alla stessa memoria fisica della copia principale (nessuna copia immediata).
- Quando il processo tenta di **scrivere** nella memoria mappata, il sistema operativo:
  1. alloca un nuovo blocco di memoria fisica;
  2. copia il contenuto dalla copia principale al nuovo blocco (*copy-on-write*);
  3. reindirizza la memoria virtuale del processo verso il nuovo blocco fisico.
- Le modifiche **non vengono propagate** al file sottostante.

